

Recherche de documents « Document Retrieval »

Séance 5

Mesure de similarité et recherche de documents pertinents

I. Introduction :

Après avoir préparé le corpus (nettoyage, filtrage lexical, normalisation morphologique) et construit différentes représentations vectorielles pour les tokens (BoW, TF, TF-IDF, BM25, n-grammes), l'étape suivante consiste à exploiter ces représentations pour rechercher les documents ou phrases les plus pertinents par rapport à une requête. Elle introduit également la construction de représentations vectorielles adaptées à différents niveaux de granularité (phrase, document, corpus), nécessaires pour effectuer des recherches à plusieurs échelles.

Cette séance a pour objectif de :

- Comprendre les principes fondamentaux de la similarité entre vecteurs.
- Formuler une requête utilisateur.
- Implémenter différentes mesures de similarité et de distance.
- Développer un moteur de recherche simple, capable de traiter :
 - des requêtes sous forme de phrases ou de documents longs ;
 - des bases de recherche composées de vecteurs de phrases ou de vecteurs de documents.
- Évaluer l'impact des configurations de prétraitement et des différents descripteurs vectoriels sur les résultats de recherche.
- Mettre en place des expérimentations modulaires permettant d'explorer la recherche dans des sous-corpus, dans le corpus global ou selon la langue.
- Expérimenter, éventuellement, avec des **requêtes bilingues ou croisées**.
- Préparer les résultats pour une visualisation qui facilitera l'analyse qualitative, notamment en l'absence d'annotations.
- Développer les fonctions de recherche et les visualisations qui serviront de base pour l'intégration future dans une interface Homme-machine (atelier 6).

II. Plan de la cinquième étape de l'atelier :

1. Vectorisation des requêtes et du corpus

Dans cette séance, nous allons préparer les représentations vectorielles nécessaires au calcul de similarité entre une requête (phrase ou document) et les éléments d'un corpus (phrases ou documents).

Toutes les fonctions de vectorisation utiliseront les descripteurs textuels définis dans la séance précédente (*Bag-of-Words*, *TF*, *TF-IDF*, *BM25*, *etc.*).

Comme les scénarios de recherche documentaire sont variés, nous devons être capables de vectoriser :

- **une phrase seule** (cas des requêtes courtes)
- **un document entier** (cas des requêtes plus longues ou de la comparaison document–document)
- **un corpus complet**, au niveau :
 - des **phrases**,
 - des **documents**,
 - et éventuellement de documents reconstruits par **agrégation des vecteurs des phrases**.

L'objectif de cette section est donc de fournir un ensemble de fonctions générales permettant, à partir des encodeurs déjà implémentés, de produire les vecteurs correspondant aux différents niveaux de granularité. **Un vecteur est représenté par une liste Python ou un vecteur Numpy.**

a. Vectorisation d'une phrase

Une phrase est représentée comme une liste de mots (tokens, racines, lemmes).

- Écrire une fonction **vectoriser_phrase (phrase, vocabulaire_direct, methode="tfidf", **params)** qui transforme une phrase (déjà tokenisée sous forme de liste de mots) en un vecteur numérique selon une méthode de pondération choisie parmi les méthodes vues à la séance 4 (par défaut : TF-IDF). Cette fonction réutilisera les fonctions de calcul définies à la séance précédente. Le paramètre « ****params** » permet de transmettre les paramètres supplémentaires requis par certaines méthodes (par exemple les valeurs IDF, etc.).

b. Vectorisation d'un document

Un document est une liste de phrases, et chaque phrase est une liste de mots. Nous proposons deux stratégies de vectorisation.

i. Vectorisation basée sur les mots du document

On considère toutes les phrases du document comme un seul texte (concaténation des mots).

- Écrire une fonction **vectoriser_document_mots (document, vocabulaire_direct, methode="tfidf", **params)** qui transforme un document (déjà tokenisée sous forme d'une liste de listes de mots) en un vecteur numérique selon une méthode de pondération choisie parmi les méthodes vues à la séance 4 (par défaut : TF-IDF). Cette fonction va aplatir toutes les phrases en une seule liste de mots et fera appel à la fonction **vectoriser_phrase (phrase, vocabulaire_direct, methode="tfidf", **params)** pour calculer le vecteur.

ii. Vectorisation par agrégation des vecteurs des phrases

Un document est une liste de phrases, on vectorise chaque phrase séparément, puis on combine ces vecteurs en un seul selon une stratégie d'agrégation ("**moyenne**", "**somme**", "**max**").

- Écrire une fonction **vectoriser_document_agrege** (**document**, **vocabulaire_direct**, **methode="tfidf"**, **strategie = "moyenne"**, ****params**) qui transforme un document (déjà tokenisée sous forme d'une liste de listes de mots) en un vecteur numérique selon une méthode de pondération choisie parmi les méthodes vues à la séance 4 (par défaut : TF-IDF). Cette fonction fera appel à la fonction **vectoriser_phrase** (**phrase**, **vocabulaire_direct**, **methode="tfidf"**, ****params**) pour vectoriser chacune des phrases. Ensuite les vecteurs obtenus sont agrégés selon la stratégie passée en paramètre.

c. Vectorisation du corpus

Le corpus a été représenté par un dictionnaire lors de la séance 3 :

```
{id_doc1 : [[mot11, mot12,..., mot1f], ..., [motk1, motk2,..., motks]],
 id_doc2 : [[mot11, mot12,..., mot1p], ..., [motm1, motm2,..., motmq]],
 ...
 id_docN : [[mot11, mot12,..., mot1y], ..., [motw1, motw2,..., motwz]]}
```

Nous voulons pouvoir vectoriser le corpus selon trois niveaux :

Niveau	Description	Structure retournée
"phrase"	un vecteur pour chaque phrase	{id_doc : [vect_phrase_1, ..., vect_phrase_n]}
"document_mots"	un vecteur par document, basé sur les mots concaténés	{id_doc : vecteur}
"document_agrege"	un vecteur par document, basé sur l'agrégation des phrases	{id_doc : vecteur}

- Écrire une fonction générique **vectoriser_corpus**(**corpus**, **vocabulaire_direct**, **methode="tfidf"**, **niveau="document_mots"**, **strategie_agregation="moyenne"**, ****params**) qui transforme un corpus en une suite de vecteurs selon le niveau passé en paramètre. Cette fonction fera appel aux fonctions précédentes.

d. Synthèse des scénarios traités

N°	Requête	Corpus (éléments comparés)	Paramètre niveau à passer à vectoriser_corpus
1	phrase	phrases d'un document	phrase
2	phrase	phrases de tous les documents	phrase
3	phrase	documents_mots	document_mots
4	phrase	documents_agrege	document_agrege
5	document_mots	documents_mots	document_mots
6	document_agrege	documents_agrege	document_agrege
7	document_agrege	documents_mots	document_mots
8	document_mots	documents_agrege	document_agrege

Grâce aux fonctions de vectorisation définies ci-dessus, on peut traiter différents scénarios selon les combinaisons suivantes :

e. Construction des métadonnées du corpus

Pour faciliter la recherche et l'analyse des documents vectorisés, il est indispensable de disposer d'un dictionnaire centralisant les informations essentielles de chaque document. Ces métadonnées permettront de lier chaque vecteur à son contexte (langue, sous-corpus, étudiant, etc.) et seront utilisées pour le classement, l'affichage et la visualisation des résultats. Le dictionnaire `meta_corpus` doit contenir pour chaque document les informations suivantes :

- `id_doc` : identifiant unique du document (ex. `etu01_fr`) ;
 - `langue` : langue du document (fr ou en) ;
 - `sous_corpus` : catégorie d'origine (`UFR`, `IUT`, `Univ_Nationale`, `Univ_Etrangere`, ou `Cas_1_Phrases`) ;
 - `chemin` : chemin complet vers le fichier texte correspondant ;
 - `position` : index du document dans le corpus pour conserver un ordre stable ;
 - `titre` : nom du fichier sans extension, utilisé pour l'affichage.
 - `autres_metadonnees` : toute information utile supplémentaire (taille du document, date de création si disponible, nombre de phrases, nombre de mots, etc.)
- Écrire une fonction `construire_meta_corpus(chemin_base)` qui parcourt le corpus et retourne un dictionnaire `meta_corpus` contenant pour chaque document, ses métadonnées. Le dictionnaire retourné pourra avoir la structure suivante :

```
meta_corpus = {  
  
    'etu01_fr': {'sous_corpus': 'UFR', 'langue': 'fr', 'chemin':  
                'Corpus_Textes/Cas_2_Textes_Complets/UFR/etu01_fr.txt', ...},  
  
    'etu01_en': {'sous_corpus': 'UFR', 'langue': 'en', 'chemin':  
                'Corpus_Textes/Cas_2_Textes_Complets/UFR/etu01_en.txt', ...}, ...}
```

2. Mesures de similarité entre vecteurs

Lorsqu'on compare des vecteurs représentant des documents, des phrases ou des embeddings, plusieurs mesures de distance et similarité sont possibles. Chaque mesure possède ses avantages, limites et domaines d'application.

a. Distance Euclidienne (L2):

La distance Euclidienne mesure la distance « droite » entre deux vecteurs dans un espace n-dimensionnel. Elle est très intuitive mais sensible à l'échelle des variables.

$$Dis_{L2}(v1, v2) = \sqrt{\sum_{i=1}^n (v1_i - v2_i)^2}$$

i. Points de vigilance

- Minimum = 0 si les vecteurs sont identiques.
 - **Sensible à la magnitude**, une normalisation est souvent nécessaire (norme L2, standardisation).
 - Peut être biaisée si certaines dimensions varient davantage que d'autres.
- Écrire une fonction **calcul_distance_euclidienne(v1, v2)** qui calcule et retourne la distance Euclidienne entre deux vecteurs.

b. Distance Manhattan (L1) :

La distance Manhattan somme des écarts absolus sur chaque dimension. Elle est moins sensible aux valeurs extrêmes que la distance Euclidienne.

$$Dis_{L1}(v1, v2) = \sum_{i=1}^n |v1_i - v2_i|$$

i. Points de vigilance

- Plus robuste aux valeurs aberrantes.
 - Adaptée aux données clairsemées « sparse ».
 - Interprétation intuitive : distance dans une grille.
- Écrire une fonction **calcul_distance_manhattan(v1, v2)** qui calcule et retourne la distance Manhattan entre deux vecteurs.

c. Distance de Minkowski (p-distance)

C'est une généralisation des distances Euclidienne (p=2) et Manhattan (p=1). Elle permet d'ajuster l'importance des écarts élevés via le paramètre p.

$$Dis_{Lp}(v1, v2) = \sqrt[p]{\sum_{i=1}^n (v1_i - v2_i)^p}$$

i. Points de vigilance

- $p \geq 1$ (sinon ce n'est plus une distance au sens mathématique).
 - Plus p est élevé, plus la distance amplifie les grandes différences.
- Écrire une fonction **calcul_distance_minkowski(v1, v2, p)** qui calcule et retourne la distance Minkowski d'ordre « p » entre deux vecteurs.

d. Distance de Tchebychev (∞ -distance) :

Elle mesure l'écart maximal entre deux vecteurs. Cette distance est utile lorsque la différence la plus importante prévaut sur les autres.

$$Dis_{L\infty}(v1, v2) = \max_{1 \leq i \leq n} |v1_i - v2_i|$$

i. Points de vigilance

- Ignore toutes les autres dimensions sauf la pire.
- Utilisable pour des contraintes strictes.
- Écrire une fonction **calcul_distance_tchebychev(v1, v2)** qui calcule et retourne la distance Tchebychev entre deux vecteurs.

e. Distance de Bray-Curtis :

Elle mesure la dissimilarité en comparant les différences relatives entre composantes. Elle est très utilisée en écologie ou pour des données de fréquence.

$$Dis_{BC}(v1, v2) = \frac{\sum_{i=1}^n |v1_i - v2_i|}{\sum_{i=1}^n (v1_i + v2_i)}$$

i. Points de vigilance

- Défini uniquement si le dénominateur > 0.
- Valeur entre 0 (identiques) et 1 (complètement différents).
- Ne respecte pas la propriété de triangle → ce n'est pas une distance métrique stricte.
- Écrire une fonction **calcul_distance_bray_curtis(v1, v2)** qui calcule et retourne la distance Bray-Curtis entre deux vecteurs.

f. Similarité cosinus :

La similarité cosinus mesure l'angle entre deux vecteurs, indépendamment de leur norme : plus il est petit, plus les vecteurs se ressemblent. Elle est recommandée pour TF-IDF, les embeddings et BM25.

$$Sim_{cos}(v1, v2) = \frac{v1 \cdot v2}{\|v1\|_2 \cdot \|v2\|_2}$$

i. Points de vigilance

- Valeur entre 0 (vecteurs orthogonaux) et 1 (même direction).
- Nécessite normes non nulles.
- Insensible à la longueur : utile pour comparer des directions.
- Écrire une fonction **calcul_similarite_cosinus(v1, v2)** qui calcule et retourne la similarité cosinus entre deux vecteurs.

g. La distance cosinus

Elle est calculée à partir de la similarité cosinus.

$$Dis_{cos}(v1, v2) = 1 - Sim_{cos}(v1, v2)$$

i. Points de vigilance

- Très utilisée pour classer ou trier par similarité.
- Écrire une fonction **calcul_distance_cosinus(v1, v2)** qui calcule et retourne la distance cosinus entre deux vecteurs.

h. Similarité et distance de Jaccard

Le coefficient Jaccard compare des ensembles ou des vecteurs binaires (présence/absence). Cette similarité est excellente pour les modèles BoW binaires.

$$Sim_{jaccard}(v1, v2) = \frac{|v1 \cap v2|}{|v1 \cup v2|}$$

Pour les vecteurs non binaires, il est possible d'utiliser la version généralisée :

$$Sim_{jaccard}(v1, v2) = \frac{\sum_{i=1}^n \min(v1_i, v2_i)}{\sum_{i=1}^n \max(v1_i, v2_i)}$$

i. Points de vigilance

- Privilégier les vecteurs **binaires** pour la version classique.
- Pour des pondérations → utiliser la version généralisée.
- Écrire une fonction **calcul_similarite_jaccard(v1, v2)** qui calcule la similarité de Jaccard entre deux vecteurs.

La distance de Jaccard est calculée à partir de la similarité de Jaccard.

$$Dis_{jaccard}(v1, v2) = 1 - Sim_{jaccard}(v1, v2)$$

- Écrire une fonction **calcul_distance_jaccard(v1, v2)** qui calcule la distance de Jaccard entre deux vecteurs.

i. Distance de Hamming

Elle compte le nombre de positions dans lesquelles les deux vecteurs diffèrent. Elle est très utilisée pour comparer les vecteurs binaires, adaptée pour les modèles BoW binaires.

$$Dis_H(v1, v2) = \sum_{i=1}^n \mathbf{1}(v1_i \neq v2_i)$$

Où $\mathbf{1}(\cdot)$ est la fonction indicatrice, qui vaut :

$$\mathbf{1}(v1_i \neq v2_i) = \begin{cases} 1, & \text{si } v1_i \neq v2_i \\ 0, & \text{sinon} \end{cases}$$

i. Points de vigilance

- Pertinent uniquement pour données discrètes / binaires.
- Écrire une fonction **calcul_distance_hamming(v1, v2)** qui calcule la distance de Hamming entre deux vecteurs .

j. La distance de Jensen-Shannon

La divergence de Jensen-Shannon mesure la divergence entre distributions de probabilité. C'est une version symétrique et plus stable de la divergence de Kullback–Leibler. La distance Jensen-Shannon est égale à la racine carrée de la divergence de Jensen-Shannon.

$$Dis_{JS}(v1, v2) = \sqrt{\frac{1}{2}(Div_{KL}(v1 \parallel m) + Div_{KL}(v2 \parallel m))} \quad \text{où } m = \frac{v1 + v2}{2}$$

Et

$$Div_{KL}(v1 \parallel v2) = \sum_{i=1}^n v1_i \log\left(\frac{v1_i}{v2_i}\right)$$

i. Points de vigilance

- Les vecteurs doivent être des distributions : $\sum_{i=1}^n v_i = 1$, $v_i > 0$.
- Stable, toujours définie, bornée entre 0 et 1.
- La base du log peut être « e » (ln) ou « 2 » ($\log_2$).
- Écrire une fonction **calcul_distance_jensen_shannon(v1, v2)** qui calcule la distance de Jensen-Shannon entre deux vecteurs .

3. Recherche documentaire par requête

La recherche consiste à comparer une requête (phrase ou document) à un corpus vectorisé à différents niveaux (phrases, documents_mots, documents_agrege).

L'objectif est de calculer des scores de similarité ou de distance afin d'identifier les éléments du corpus les plus pertinents.

a. Représentation de la requête

Une requête est toujours représentée par un seul vecteur, mais la manière de le construire dépend :

1. du **type de la requête** (phrase ou document)
2. du **descripteur utilisé** (BoW, TF, TF-IDF, BM25, n-grammes, etc.)
3. du **vocabulaire et des prétraitements du corpus**, qui doivent impérativement être les mêmes :
 - mêmes étapes de nettoyage et de normalisation,

- même segmentation (tokens ou n-grammes),
- même vocabulaire indexé,
- même modèle de pondération.

Ainsi, la requête devient un vecteur strictement compatible en dimension et en structure avec les vecteurs du corpus. La requête n'est pas nécessairement vectorisée avec la *même granularité* que les documents du corpus. On peut comparer une **phrase** avec un **document_mots** ou un **document_agrege**, du moment que le descripteur et le vocabulaire sont identiques.

Pour assurer cette cohérence, on introduit deux fonctions à écrire.

- Écrire une fonction **pretraiter_requete(texte_requete, config_pretraitement)** qui applique les mêmes étapes de prétraitement que celles utilisées pour le corpus (nettoyage, normalisation, segmentation, tokenisation, etc.). Elle retourne la requête sous une forme prétraitée compatible avec les fonctions de vectorisation.
- Écrire une fonction **vectoriser_requete(texte_pretraite, vocabulaire, methode="tfidf", type_requete="phrase")** qui transforme la requête prétraitée en un vecteur unique selon la nature de la requête :
 - si **type_requete="phrase"**, elle fera appel à **vectoriser_phrase**,
 - si **type_requete="document_mots"**, elle fera appel à **vectoriser_document_mots**
 - si **type_requete="document_agrege"**, elle fera appel à **vectoriser_document_agrege**.

Elle retourne le vecteur final représentant la requête.

b. Calcul de similarité avec la requête

Une fois vectorisée, la requête est comparée à chaque élément du corpus (vecteur de phrase ou de document) pour calculer un score de similarité ou de distance selon la mesure de similarité choisie (cosinus, Jaccard, Euclidienne, etc.). La compatibilité des vecteurs est assurée par la vectorisation effectuée lors de la première section : mêmes prétraitements, même vocabulaire et même méthode de pondération. Chaque élément du corpus peut être :

- un vecteur de phrase,
- un vecteur de **document_mots**,
- un vecteur de **document_agrege**.

- Écrire une fonction **calculer_similarite(vect_requete, vect_corpus, mesure='cosinus')** qui calcule la similarité ou distance entre le vecteur unique de la requête et un vecteur unique du corpus. Elle retourne une valeur numérique représentant la similarité ou la distance.
- Écrire une fonction **calculer_scores_requete(vect_requete, corpus_vecteurs, mesure='cosinus', niveau='document_mots')** qui calcule les scores entre une requête et tous les éléments d'un corpus vectorisé sous forme de dictionnaire (**{id_doc: vecteur}** ou **{id_doc:[vecteurs_de_phrases]}**). Elle retourne un dictionnaire **{id_doc: score}** ou **{id_doc: [score_phrase1,**

`score_phrase2, ...}]` selon l'argument `niveau` $\in$ [`'phrase'`, `'document_mots'`, `'document_agrege'`].

c. Classement et affichage des résultats

Une fois les scores calculés avec la fonction `calculer_scores_requete`, il est nécessaire de trier les éléments du corpus et d'extraire les k meilleurs résultats afin d'identifier rapidement les documents ou phrases les plus pertinents pour une requête.

- Selon le niveau du corpus (`phrase`, `document_mots`, `document_agrege`), le résultat affiché peut être :
 - une phrase (scénarios 1 et 2)
 - un document (scénarios 3 à 8)
- Le moteur de recherche doit être capable de :
 - gérer plusieurs niveaux de granularité,
 - sélectionner automatiquement le bon corpus,
 - produire des résultats homogènes en fonction du choix de l'utilisateur.
- Écrire une fonction **`extraire_top_k(scores_dict, k=5, niveau='document')`** qui sélectionne les k éléments les mieux classés selon les scores calculés.
 - Pour `niveau='document'` : `scores_dict` est de la forme `{id_doc : score}`, la fonction trie les documents selon leur score décroissant et retourne les k meilleurs dans le même format `{id_doc: score}`.
 - Pour `niveau='phrase'` :

Selon la manière dont les scores ont été produits, `scores_dict` peut prendre deux formes.

 - **Scénario 1 (phrases d'un seul document)** : `scores_dict` est de la forme `{id_doc : [score_ph1, score_ph2, ..., score_phF]}`, cela signifie que toutes les phrases proviennent du même document `id_doc`. La fonction retourne les k phrases les mieux classées pour ce document sous forme de dictionnaire `{(id_doc, id_phrase) : score}`.
 - **Scénario 2 (phrases de tous les documents)** : `scores_dict` est de la forme `{ id_doc1 : [score_ph1, score_ph2, ...], id_doc2 : [score_ph1, score_ph2, ...], ..., id_docN : [score_ph1, score_ph2, ...]}`. La fonction retourne les k phrases les mieux classées parmi tous les documents sous forme de dictionnaire `{(id_doc, id_phrase): score}`.
- Écrire une fonction **`afficher_metadonnees_resultat(id_doc, score, meta_corpus)`** qui affiche les informations essentielles associées à un résultat : ID du document, score de similarité, langue, position dans le corpus, sous-corpus, et toute autre métadonnée pertinente.
- Écrire une fonction **`afficher_contenu_document(id_doc, corpus_vecteurs, niveau='document_mots')`** qui affiche le texte complet correspondant à l'identifiant du document ou la phrase associée selon le niveau utilisé pour la recherche. Cette fonction ne gère ni la mise en évidence, ni les métadonnées.
- Écrire une fonction **`highlight_mots_pertinents(texte, requete, vocabulaire)`** qui met en évidence dans le texte les mots ou lemmes présents dans la requête.

Cette fonction retourne une version annotée ou colorée du texte, et peut être utilisée dans l’affichage ou dans des outils de visualisation ultérieurs. Elle doit recevoir le même vocabulaire utilisé pour vectoriser la requête et le corpus.

- Écrire une fonction **afficher_resultat_complet(id_doc, score, corpus_vecteurs, meta_corpus, afficher_texte=False, highlight=False, requete=None, vocabulaire=None)** qui affiche les métadonnées du résultat puis, si demandé :
 - le texte associé (phrase ou document),
 - une version mise en évidence des mots pertinents si l’option highlight est activée.

Cette fonction sert d’orchestrateur et permet d’obtenir un affichage simple ou détaillé selon les besoins de l’utilisateur.

- Écrire une fonction **afficher_resultats_top_k(top_k, corpus_vecteurs, meta_corpus, afficher_texte=False, highlight=False, requete=None, vocabulaire=None)** qui parcourt les k meilleurs éléments retournés par la recherche et, pour chacun, appelle :
 - **afficher_metadonnees_resultat()** pour afficher les informations principales ;
 - **afficher_resultat_complet()** si l’utilisateur souhaite afficher le texte ou activer la mise en évidence des mots pertinents.

4. Évaluation et visualisation

Un système de recherche doit être évalué en fonction de la cohérence des scores obtenus, de la pertinence relative des éléments retrouvés et de la structure interne du corpus. Dans un corpus non annoté, il n’est pas possible de calculer des mesures classiques comme la précision ou le rappel. L’évaluation repose donc sur des outils exploratoires et des visualisations permettant d’interpréter les résultats du moteur de recherche.

a. Analyse des Top-k résultats

Une fois le pipeline de recherche appliqué (vectorisation → comparaison → tri → top-k), il est nécessaire d’examiner la cohérence des résultats obtenus.

- Écrire une fonction **filtrer_par_score(top_k, seuil=0.5)** qui ne conserve que les éléments dont le score supérieur ou égal au seuil choisi.
- Écrire une fonction **resume_top_k(top_k, meta_corpus)** qui génère un tableau récapitulatif des résultats top-k : ID du document, langue, sous-corpus d’origine, score, position éventuelle.

b. Distribution des scores

La distribution des scores permet d’évaluer la discrimination d’une requête : si les scores sont concentrés, la requête est peu sélective ; s’ils sont dispersés, elle distingue clairement les documents.

- Écrire une fonction **distribution_scores(scores)** qui affiche la distribution des scores sous forme d'histogramme ou boxplot.
- Écrire une fonction **stats_scores(scores)** qui retourne min, max, moyenne, médiane et quartiles.
- Écrire une fonction **comparer_distributions_requetes(scores_requetes)** qui affiche sur un même graphique les distributions de plusieurs requêtes.
- Écrire une fonction **normaliser_scores(scores, methode='minmax')** qui normalise les scores pour permettre les comparaisons entre requêtes ou modèles.

c. Répartition par sous-corpus et langue

Une bonne requête produit un top-k cohérent, localisé dans certains sous-corpus ou langues. La répartition du top-k permet donc d'évaluer le comportement du moteur de recherche.

- Écrire une fonction **analyse_sous_corpus(top_k, meta_corpus)** qui retourne la répartition des documents top-k par sous-corpus.
- Écrire une fonction **repartition_pourcentage(top_k, meta_corpus)** qui retourne la proportion de documents de chaque sous-corpus et langue.
- Écrire une fonction **afficher_repartition_barplot(repartition)** qui visualise ces proportions sous forme d'histogramme.
- Écrire une fonction **filtrer_top_k_par_categorie(top_k, meta_corpus, categorie='langue', valeur='fr')** qui sélectionne les éléments correspondant à une catégorie spécifique.

d. Matrices de similarité et heatmaps :

La matrice de similarité permet une exploration globale du corpus : documents proches, documents isolés, zones de regroupements.

- Écrire une fonction **matrice_similarite(corpus_vecteurs, mesure='cosinus')** qui calcule une matrice de similarité (ou de distance) entre tous les documents du corpus. Chaque cellule [i,j] représente la similarité (ou distance) entre le document i et le document j.
- Écrire une fonction **afficher_heatmap_similarite(matrice, labels=None, titre='Heatmap de similarité')** qui visualise la matrice sous forme de carte thermique (heatmap).
 - Les axes peuvent être annotés avec les IDs des documents pour faciliter l'interprétation.
 - Vous pouvez observer visuellement quels documents sont proches ou éloignés les uns des autres.
- Écrire une fonction **matrice_distance_normalisee(matrice_similarite)** qui transforme les similarités en distances normalisées (0-1) pour comparaison inter-corpus.
- Écrire une fonction **top_paires_similaires(matrice, top=5)** qui identifie les documents les plus proches dans le corpus.

- Écrire une fonction **top_paires_différentes(matrice, top=5)** qui identifie les documents les plus éloignés dans le corpus.
- Écrire une fonction **heatmap_sous_corpus(matrice, meta_corpus, sous_corpus='UFR')** qui affiche la heatmap pour un sous-corpus particulier afin d'observer sa cohérence interne.

5. Expérimentations exploratoires

Cette partie vise à explorer l'impact des différentes configurations de prétraitement et des choix de représentation vectorielle sur les résultats de recherche documentaire. L'objectif est de mettre en évidence des tendances et comportements dans un corpus non annoté, afin de comprendre comment chaque paramètre influence les résultats et la qualité du classement des documents.

a. Prétraitement global du corpus pour l'expérimentation exploratoire

Pour certaines expériences, comme la comparaison de prétraitements, de descripteurs ou de tailles de n-grammes, il est essentiel d'avoir un **corpus uniformément prétraité**. Cela garantit que toute variation observée dans les résultats provient uniquement des choix expérimentaux, et non de différences dans le nettoyage ou la normalisation.

- Écrire une fonction **pretraiter_corpus(corpus, config_pretraitement)** qui applique le prétraitement sur l'ensemble du corpus selon une configuration donnée. Le paramètre « **corpus** » est un dictionnaire de liste de listes et le paramètre « **config_pretraitement** » est un dictionnaire dans lequel vous allez définir tous les paramètres de prétraitement dont vous aurez besoin. Cette fonction retourne un corpus prétraité ayant la même structure que le corpus initial.

b. Impact des 5 configurations de prétraitement, filtrage et normalisation

Le prétraitement du texte joue un rôle déterminant sur les résultats de recherche documentaire. Cette étape permet d'étudier comment différentes configurations combinant filtrage et normalisation morphologique influencent la structure lexicale du corpus et les performances du moteur de recherche.

Voici les cinq configurations à tester (séance 3).

Config	Stopwords	Longueur min	Non alphabétiques	Stemming	Lemmatisation
A	✗	✗	✗	✗	✗
B	✓	✓	✓	✗	✗
C	✓	✓	✓	✓	✗
D	✓	✓	✓	✗	✓

Config	Stopwords	Longueur min	Non alphabétiques	Stemming	Lemmatisation
E	✓	✓	✓	✓	✓

- Écrire une fonction **tester_pretraitements(requete, corpus, configs, vocabulaire)** qui applique différentes configurations de prétraitement et retourne les top-k résultats et les statistiques du vocabulaire pour chaque configuration.

Comparer la taille du vocabulaire, le bruit lexical et les résultats top-k pour une même requête et observer l'évolution du top-k documents selon chaque configuration.

c. Impact du type de représentation

Différents descripteurs vectoriels (BoW, TF, TF-IDF, BM25, n-grammes) peuvent produire des résultats très différents. Cette sous-section permet d'observer l'effet du choix de représentation sur le classement des documents et la distribution des scores, afin d'identifier le descripteur le plus adapté à la tâche.

- Écrire une fonction **tester_descripteurs(requete, corpus, descripteurs)** qui compare les top-k résultats obtenus avec différents descripteurs.

d. Impact des n-grammes

Les vocabulaires et descripteurs peuvent être construits à différentes granularités, en capturant des unités de contexte croissant : unigrammes (mot isolé), bigrammes (séquences de deux mots), trigrammes (séquences de trois mots), etc. Cette partie explore l'effet de la taille du contexte sur la capacité du moteur à distinguer des documents proches ou similaires, et à produire des top-k plus discriminants.

- Écrire une fonction **tester_ngrams(requete, corpus, n_list, vocabulaire)** qui itérera sur chaque valeur de `n` dans `n_list`, construira le vocabulaire et les vecteurs correspondants, puis retournera le top-k des documents pour chaque taille de n-gramme.

e. Impact de la métrique de similarité

La mesure de similarité utilisée pour comparer vecteurs de documents influence directement le classement et la cohérence des résultats. Cette sous-section vise à comparer différentes métriques (cosinus, Euclidienne, Manhattan, Jaccard, etc.) et à identifier celles qui produisent des regroupements plus pertinents par sous-corpus ou par langue.

- Écrire une fonction **comparer_distances(corpus, mesures)** qui compare plusieurs mesures de similarité ou de distance (cosinus, Euclidienne, Manhattan, Jaccard, etc.) sur le même corpus.

Identifier les mesures qui produisent des regroupements cohérents par sous-corpus ou langue et observer comment la métrique influence la sélection des documents dans le top-k.

f. Échelle de la recherche cas requête phrase vs requête document

L'échelle de la requête influence fortement la granularité et la précision des résultats de recherche documentaire. Plusieurs dimensions doivent être considérées pour évaluer l'impact de cette échelle sur les top-k obtenus et la discrimination des documents :

i. Type de requête : phrase ou document

- Une **phrase courte** permet de tester des correspondances lexicales locales et d'identifier des phrases précises ou des passages pertinents dans le corpus.
- Un **document complet** comme requête capture un contexte plus large, ce qui peut produire des top-k plus stables et cohérents au niveau des documents.

ii. Granularité du corpus cible

- Une requête phrase peut être comparée à :
 - ⇒ **Toutes les phrases du corpus**, pour détecter des passages similaires.
 - ⇒ **Tous les documents**, soit en concaténant les mots de chaque document, soit en agréant les vecteurs de leurs phrases.

iii. Construction des représentations documentaires

Les documents peuvent être représentés par :

- **Concaténation de tous les mots** du document avant calcul du descripteur, pour capturer un contexte global.
- **Agrégation des vecteurs des phrases**, par moyenne, somme ou maximum, pour conserver la structure interne et la distribution des informations.
- Écrire une fonction **comparer_echelle_requete (requete, corpus_vecteurs, type_requete='phrase', niveau_corpus='document_agrege', mesures=['cosinus'], top_k=5)** qui compare les résultats top-k selon l'échelle de la requête et la granularité du corpus.
 - type_requete : 'phrase' ou 'document'
 - niveau_corpus : 'phrase', 'document_mots', 'document_agrege'
 - mesures : liste de métriques de similarité ou distance
 - top_k : nombre de résultats les plus pertinents à extraire
- Comparer les top-k obtenus pour une requête courte vs un document complet.
- Identifier les combinaisons requête/granularité du corpus produisant les résultats les plus discriminants.
- Évaluer l'effet de l'agrégation vs la concaténation sur la pertinence des top-k.
- Observer la sensibilité des métriques de similarité aux différentes échelles de requêtes et de représentations.

g. Impact de la langue

Dans un corpus bilingue, la langue des documents influence directement les résultats de la recherche documentaire, même lorsque l'on utilise des descripteurs basés sur les fréquences de mots (BoW, TF, TF-IDF, BM25). Les variations lexicales entre le français et l'anglais peuvent modifier les top-k et la cohérence des regroupements. Cette sous-section explore deux aspects complémentaires : la comparaison mono-langue et la recherche combinée multi-langue.

i. Comparaison des résultats entre langues (mono-langue)

L'objectif est de vérifier si une requête en français et sa traduction en anglais produisent des résultats similaires lorsqu'elles sont appliquées à leurs corpus respectifs. Expérimenter au niveau phrase ou document complet, en utilisant les mêmes méthodes de vectorisation et stratégies d'agrégation que pour les analyses monolingues.

- Écrire une fonction **top_k_mono_langue(requete, corpus, niveau_corpus = 'document_agrege', mesure='cosinus', top_k=5)** qui vectorise la requête dans la même langue que le corpus et calcule les scores de similarité ou de distance entre la requête et tous les documents/phrases du corpus. La fonction retourne un dictionnaire {id_doc: score} ou {(id_doc, id_phrase): score} avec les top-k résultats.
- Écrire une fonction **comparer_top_k_langues(top_k_fr, top_k_en)** qui compare les top-k obtenus pour la requête française et sa traduction anglaise et indique le nombre de documents identiques, les différences de classement et les écarts dans les scores.

ii. Recherche combinée par fusion des top-k

L'objectif est d'étudier l'effet de la fusion des résultats obtenus pour une requête et sa traduction, afin de produire un top-k global intégrant les deux langues.

- Effectuer deux recherches séparées : une sur le corpus français (**top_k_fr**) et une sur le corpus anglais (**top_k_en**).
 - Normaliser les scores si nécessaire.
 - Fusionner les résultats pour produire un top-k global basé sur les scores combinés.
 - Comparer ce top-k combiné avec les résultats mono-langue pour observer l'effet sur la couverture et la cohérence.
-
- Écrire une fonction **recherche_multilingue_fusion(requete_fr, requete_en, corpus_fr, corpus_en, niveau_corpus='document_agrege', mesure='cosinus', top_k=5)** qui effectue les recherches séparées pour la requête et sa traduction et fusionne les scores obtenus (par exemple par moyenne ou somme). La fonction retourne le top-k global combiné sous forme d'un dictionnaire {id_doc: score} ou {(id_doc, id_phrase): score}.
 - Écrire une fonction **comparer_top_k_mono_vs_fusion(top_k_mono, top_k_fusion)** qui compare les résultats obtenus avec une recherche mono-

langue et ceux obtenus après fusion et analyse les changements dans le classement, l'augmentation de la couverture et la cohérence entre les sous-corpus.

h. Impact du corpus : Recherche locale ou globale

La portée du corpus utilisé pour la recherche influence directement les résultats et leur pertinence. Une recherche locale se limite à un sous-corpus spécifique (par exemple, les documents d'un parcours de formation particulier : UFR, IUT, Univ_Nationale, Univ_Etrangere), tandis qu'une recherche globale s'effectue sur l'ensemble du corpus. Cette sous-section explore l'impact de cette granularité sur le classement des documents et la cohérence des résultats.

i. Recherche locale par sous-corpus

L'objectif est d'identifier les documents les plus pertinents au sein d'un sous-corpus donné. Cela permet de comparer des documents homogènes selon l'origine de formation. Cette recherche locale est utile pour détecter les similarités internes et les variations stylistiques ou thématiques spécifiques à un groupe.

- Écrire une fonction **top_k_local(requete, corpus_sous_corpus, niveau_corpus='document_agrege', mesure='cosinus', top_k=5)** qui applique la vectorisation et le calcul de similarité au sous-corpus choisi. La fonction retourne les top-k documents ou phrases les plus pertinents dans ce sous-corpus.
- Écrire une fonction **analyse_repartition_local(top_k, meta_corpus)** qui analyse la distribution des documents top-k au sein du sous-corpus et permet d'identifier si certains documents dominent la recherche locale.

ii. Recherche globale sur l'ensemble du corpus

L'objectif de cette section est d'identifier les documents les plus pertinents dans le corpus complet, toutes origines confondues. Cette approche permet d'observer les regroupements inter-parcours ou inter-langues et offre une vue plus complète sur la distribution des scores et la diversité des documents retrouvés.

- Écrire une fonction **top_k_global(requete, corpus_complet, niveau_corpus='document_agrege', mesure='cosinus', top_k=5)** qui applique la recherche sur tous les sous-corpus combinés. Elle retourne les top-k documents ou phrases les plus pertinents du corpus total.
- Écrire une fonction **analyse_repartition_global(top_k, meta_corpus)** qui analyse la distribution des documents top-k par sous-corpus et par langue et permet de visualiser la cohérence globale et les éventuels biais selon l'origine des documents.

iii. Comparaison entre recherche locale et globale

L'objectif est d'étudier l'effet de la portée du corpus sur les résultats en identifiant les documents qui apparaissent uniquement en recherche globale et en observant comment la pertinence relative change lorsque l'on passe d'une recherche locale à une recherche globale.

- Écrire une fonction **comparer_local_vs_global(top_k_local, top_k_global)** qui compare les classements et scores obtenus pour une requête donnée en local et en global et met en évidence les différences dans les top-k et la couverture du corpus.
- Écrire une fonction **visualiser_diff_local_global(diff_local_global)** qui produit un graphique montrant les documents présents uniquement en recherche globale, ceux conservés en recherche locale et ceux partagés.